ALGORITMIKA & PROGRAMOZÁSI ALAPOK

A do-while Ciklus és a Hátultesztelés Logikája

Garantált egyszeri lefutás, valós életből vett kódminták, Python emuláció és ciklus-összehasonlítás

15 diás átfogó gyakorlati útmutató

ALAPELV

A Hátultesztelő Ciklus Működése

Először végrehajt

A ciklusmag utasításai a feltétel vizsgálata nélkül, kötelezően lefutnak legalább egyszer.

Utólag ellenőriz

A logikai feltétel a ciklusmag legvégén dől el: ha igaz, újraindul, ha hamis, kilép.

Minimális futási szám: 1

Ellentétben a while ciklussal, itt a mag garantáltan nem hagyható ki az első körben.

```
// Általános C / C++ / Java / C# szintaxis
do {
    // Ciklusmag: legalább egyszer lefut
    feladat_vegrehajtasa();
} while (feltetel_igaz);

print("Program folytatása...");
```

SZERKEZETI KÜLÖNBSÉG

Elöltesztelés (while) vs. Hátultesztelés (do-while)

Elöltesztelő (while)

Feltételellenőrzés a belépés ELŐTT történik.

- **Minimális futás:** 0 alkalom.
- **Kockázat:** Ha a feltétel rögtön hamis, egyszer sem fut le.
- **Használat:** Amikor nem biztos, hogy egyáltalán kell dolgozni.

Hátultesztelő (do-while)

Feltételellenőrzés a kilépés ELŐTT történik.

- **Minimális futás:** 1 alkalom.
- **Garancia:** A beolvasás vagy művelet megtörténik a vizsgálat előtt.
- **Használat:** Felhasználói adatbekérés és menük kezelése.

PYTHON SAJÁTOSSÁG

Hogyan valósítjuk meg a do-while-t Pythonban?

A Python nyelvben nem létezik natív `do-while` kulcsszó. A hátultesztelés szabványos megoldása a **while True és break** kombinációja.

1. Belépés korlátlanul

A `while True:` feltétel azonnal átengedi a végrehajtást.

2. Kilépési vizsgálat alul

A blokk alján egy `if not feltetel: break` zárja le a menetet.

```python
# Idiomatikus Python do-while minta
while True:
    # 1. Művelet elvégzése (garantáltan lefut)
    adat = input("Adj meg egy pozitív számot: ")

    # 2. Hátultesztelés: ha megfelelő, kilépünk!
    if int(adat) > 0:
        break

print(f"Elfogadva: {adat}")
```

1. GYAKORLATI KÓDPÉLDA

Konzolos Főmenü Vezérlése

```
while True:
    print("\n--- FŐMENÜ ---")
    print("1. Új játék indítása")
    print("2. Beállítások")
    print("3. Kilépés")

    valasztas = input("Válassz menüpontot (1-3): ")

    if valasztas == "3":
        print("Viszlát!")
        break  # Kilép a menüből
    elif valasztas in ["1", "2"]:
        print(f"A(z) {valasztas}. funkció betöltése...")
```

Miért do-while?

A menüt legalább egyszer meg KELL mutatni a felhasználónak, mielőtt döntenie kellene.

Nem kell kódismétlés

Hagyományos while esetén a menü kirajzolását meg kellene duplázni a ciklus előtt.

2. GYAKORLATI KÓDPÉLDA

Bankkártya PIN Ellenőrzés

```
helyes_pin = "1234"
probalkozasok = 0

while True:
    megadott = input("Kérjük adja meg a PIN kódot: ")
    probalkozasok += 1

    if megadott == helyes_pin:
        print("Sikeres azonosítás! Tranzakció indítása.")
        break
    elif probalkozasok >= 3:
        print("Kártya letiltva! Túl sok hibás próbálkozás.")
        break
```

Első próbálkozás joga

Minden felhasználó kap legalább 1 esélyt a megadásra.

Kettős termináció

A hátultesztelés vizsgálja mind a helyes adatot, mind a biztonsági limitet.

3. GYAKORLATI KÓDPÉLDA

Hőmérséklet Szenzor Mérése és Szűrése

```python
import random

while True:
    # Mérés szimulálása -30 és 60 fok között
    mert_ertek = random.randint(-30, 60)
    print(f"Mért adat: {mert_ertek} °C")

    # Hátultesztelés: megismételjük, ha mérési hiba (extrém adat) van
    if -10 <= mert_ertek <= 45:
        print("Érvényes adat rögzítve az adatbázisban.")
        break
    print("Szenzorhiba! Újramérés 1 másodperc múlva...")
```

Hardveres mintavételezés

A mérőeszköznek le kell futnia ahhoz, hogy kiderüljön: megbízható-e a jel.

Automatikus újramérés

Amíg hibás tartomány érkezik, a ciklus automatikusan újrapróbálkozik.

4. GYAKORLATI KÓDPÉLDA

Újradobás Hatost Dobva (Társasjáték Ciklus)

```python
import random

dobasok = []

while True:
    dobas = random.randint(1, 6)
    dobasok.append(dobas)
    print(f"Dobás eredménye: {dobas}")

    # Ha nem 6-ost dobott, véget ér a köre
    if dobas != 6:
        break
    print("Bónusz dobás hatos dobásért!")

print(f"Kör vége! Dobások: {dobasok}")
```

Garantált kezdeti dobás

A játékosnak kötelezően dobnia kell egyet a forduló megkezdéséhez.

Kör végi ellenőrzés

Csak a dobás megtörténte után derül ki, jár-e jutalomkör.

5. GYAKORLATI KÓDPÉLDA

Számológép "Szeretnéd újrakezdeni?" Ciklussal

```python
while True:
    a = float(input("Első szám: "))
    b = float(input("Második szám: "))
    print(f"Összeg: {a + b}")

    valasz = input("Szeretnél még egyet számolni? (i/n): ").lower()

    # Ha nem 'i', befejezzük a futást
    if valasz != "i":
        print("Köszönjük a használatot!")
        break
```

Nincs felesleges előkérdezés

A program rögtön a lényeggel indít, ahelyett hogy előre megkérdezné: "Kezdjük-e?".

Felhasználóbarát élmény

Egy számítás elvégzése után azonnal eldönthető az ismétlés.

NYELVI KÖRKÉP

Hogyan néz ki a do-while C, C++, C#, Java nyelveken?

```c
// C / C++ / C# Megvalósítás
int szam;
do {
    printf("Pozitív szám: ");
    scanf("%d", &szam);
} while (szam <= 0);
```

```java
// Java Megvalósítás
int jegy;
do {
    System.out.print("Jegy (1-5): ");
    jegy = sc.nextInt();
} while (jegy < 1 || jegy > 5);
```

ÖSSZEHASONLÍTÁS

A Három Cikluskategória Összevetése

Szempont	for Ciklus	while Ciklus	do-while Ciklus
Tesztelés helye	Elöltesztelő (léptetéskor)	Elöltesztelő (belépés előtt)	**Hátultesztelő (kilépéskor)**
Minimális lefutás	0 alkalom	0 alkalom	**Garantáltan 1 alkalom**
Lépésszám jellege	Előre ismert / véges	Előre ismeretlen	Előre ismeretlen
Fő felhasználás	Listák, range tartományok	Feltételes várás, események	Adatbekérés, menük, játékok
Python forma	`for x in lista:`	`while feltetel:`	`while True: + break`

TERVEZÉSI SZABÁLY

Döntési Segédlet Ciklusválasztáshoz

Tudod a lépésszámot?

Ismert méretű lista vagy fix számsorozat bejárása.

→ for ciklus

Lehet, hogy nem kell lefutnia?

Feltételhez kötött indítás, ahol a 0 futás is megengedett.

→ while ciklus

Muszáj egyszer lefutnia?

Bemenet kérése, menü rajzolása vagy kezdeti próba.

→ do-while ciklus

HIBAMEGELŐZÉS

Tipikus Bakik do-while Emulációkor

Kifelejtett break

Ha a kilépési if ág nem tartalmaz break utasítást, a ciklus örökre a memóriában reked.

Fordított feltétel

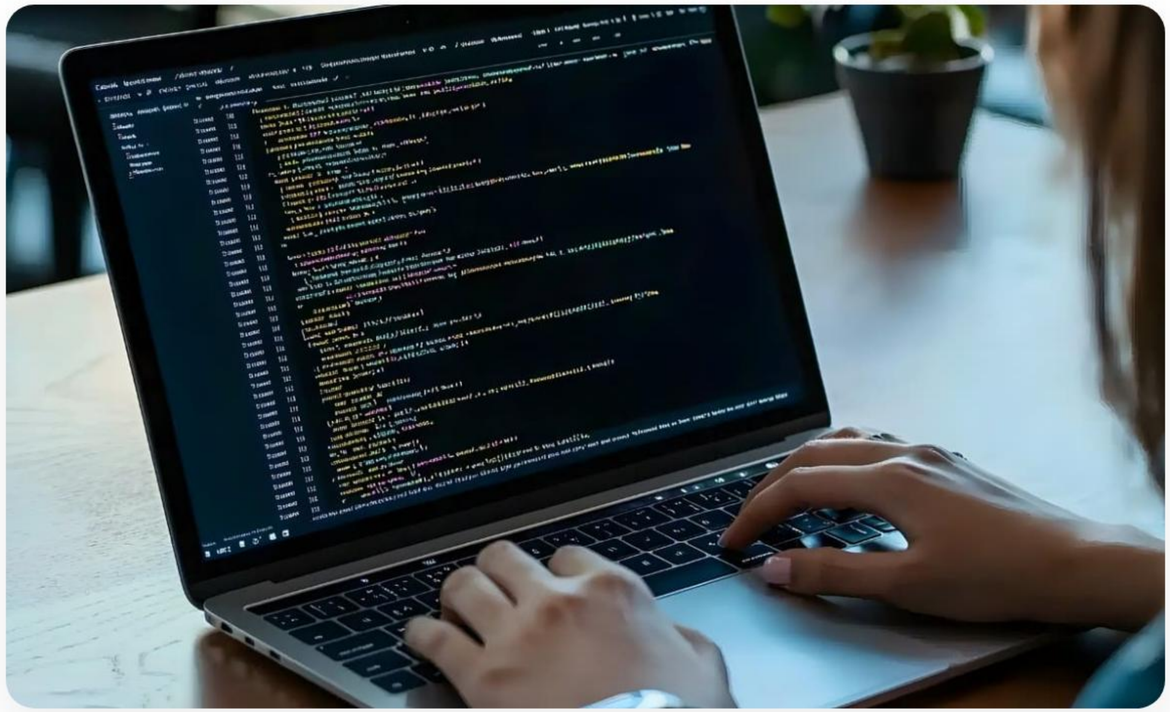
A do-while-ban a bennmaradás feltételét adjuk meg, a break-nél viszont a kilépésit.

Tiszta termináció

Mindig gondoskodj a felhasználó visszajelzéséről közvetlenül a kilépés előtt.

SZOFTVERMINŐSÉG

A Tiszta Ciklusszervezés Előnyei



DRY (Don't Repeat Yourself)

Nem kell ugyanazt az adatbekérő sort a ciklus előtt és a ciklusban is leírni.

Robusztus hibakezelés

A program garantáltan ellenőrzött adatokkal lép tovább a következő logikai lépésre.

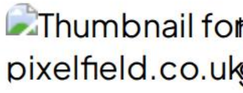


A Hátultesztelés Mesterfogásai

Használd a do-while logikát minden olyan esetben, amikor az első lépés megtétele nélkülözhetetlen a döntés meghozatalához!

#Hátultesztelés #DoWhileCiklus #Algoritmusok

Image Sources


Thumbnail for pixelfield.co.uk

https://pixelfield.co.uk/blog/wp-content/uploads/2025/05/7-best-programming-languages-for-game-development.jpg

Source: pixelfield.co.uk